autolisp-test Development Plan

Codex

June 24, 2026

Contents

1 Purpose

This document defines the plan for `autolisp-test`, a conformance-oriented AutoLISP / Visual LISP test suite.

The target outcome is a reusable test corpus that can be run against:

- `clautolisp`,
- real AutoCAD releases,
- real BricsCAD releases,
- future third-party implementations,

while keeping the reported behaviour tied to:

- implementation name,
- implementation version,
- selected host profile,
- detected platform,
- detected runtime extensions.

2 Primary Goal

`autolisp-test` tests the AutoLISP / Visual LISP language as defined by the local specification. It does not test `clautolisp`'s private behaviour, and it does not borrow from any Common Lisp test corpus.

The suite is therefore independent of the implementation under test:

- the harness is written in pure AutoLISP,
- the corpus is sourced from the local AutoLISP / Visual LISP draft specification and from executable probes against real AutoCAD and BricsCAD releases,
- `clautolisp` runs the suite as one implementation among others, with no special privileges.

It should prefer:

- vendor-documented behaviour,
- behaviour confirmed by executable probes against real implementations,
- version-qualified results where the language is historically unstable or under-specified.

It should avoid treating accidental `clautolisp` behaviour as normative.

3 Conformance Model

`autolisp-test` characterises every test by a **profile** and by **platform / runtime** tags.

3.1 Profiles

The three profiles describe the language-level conformance level a test belongs to:

`STRICT` behaviour intended to hold on AutoCAD AND BricsCAD. The strict profile is **maximal** by default: an entry sits in `STRICT` unless probe data or vendor documentation establishes a divergence.

`AUTOCAD` behaviour attested or documented on AutoCAD only.

`BRICSCAD` behaviour attested or documented on BricsCAD only.

The strict profile is therefore a lower bound on the shared standard. Every divergence between AutoCAD and BricsCAD is recorded by adding twin tests under the `AUTOCAD` and `BRICSCAD` profiles, and (when there is still a defensible common subset) keeping a narrowed `STRICT` test for the shared behaviour.

3.2 Tags

Two orthogonal axes refine applicability without inflating the profile space:

`PLATFORM-TAGS` subset of `(WINDOWS LINUX MACOS)`. Empty list means platform-agnostic.

`RUNTIME-TAGS` subset of `(COM VLA VLAX VLAX-CURVE VLR ARX BRX OBJECTDBX DCL GRAPHICS USER-INPUT EXPRESS-TOOLS DOSLIB)`. Empty list means no specific runtime extension is required.

A test is **applicable** to an implementation only if every required platform tag and runtime tag is satisfied by the detected target. When a test is not applicable, it is reported `NOT-APPLICABLE` rather than `FAIL`.

For example, the COM-bound subset of `VLAX-*` carries `RUNTIME-TAGS (COM VLAX)` and `PLATFORM-TAGS (WINDOWS)`. A `clautolisp` run on Linux will mark those tests `NOT-APPLICABLE` and still revendiquate `STRICT` conformance for the rest.

3.3 Authority

Each entry carries an authority level that records the strength of the evidence behind it:

`DOCUMENTED` backed by vendor documentation (Autodesk or Bricsys). This is the default. The level is kept distinct from `TESTED-*` because vendor documentation has been observed to diverge from the actual product behaviour.

`TESTED-AUTOCAD` confirmed by an executable probe against a named AutoCAD version.

`TESTED-BRICSCAD` confirmed by an executable probe against a named BricsCAD version.

`TESTED-BOTH` confirmed by probes against both AutoCAD and BricsCAD.

`INFERRED` derived from multiple sources or strong consistency arguments when no direct documentation or probe is available.

A `STRICT` entry should ideally be `TESTED-BOTH`. Until probes accumulate, `DOCUMENTED` is acceptable but flagged as a candidate for promotion.

3.4 Verdicts

A run produces a matrix of verdicts. For each combination of profile and required tag set, the harness reports one of:

`CONFORMS` every applicable test in that subset passed;

`DEVIATES (n failing)` at least one applicable test failed;

`NOT-APPLICABLE` no applicable test in that subset, given the detected target.

Typical reported subsets:

- `strict standard`,
- `autocad profile`,
- `bricscad profile`,
- `autocad + windows + com`,
- `bricscad + windows + com`,
- `autocad + arx`,
- `bricscad + brx`,
- `strict + objectdbx`, etc.

The matrix is the canonical output of a run.

4 Inventory

The harness ships with a machine-readable inventory of every entry defined in the local specification at:

- `autolisp-test/harness/inventory.sexp`

Initial counts (matching the current specification draft):

- Functions: 938
- Special forms: 19
- Variables: 5
- Types: 5
- Reader-syntax entries: 7
- Distinguished objects: 1

- Conditions: 1
- Total: 976 entries

Each entry is a property list with `KIND`, `NAME`, `PROFILE`, `PLATFORM-TAGS`, `RUNTIME-TAGS`, `AUTHORITY`, `SOURCE-EVIDENCE`. The default classification rules are documented in the file header.

Initial classification choices:

- `PROFILE` defaults to `STRICT` (maximal strict standard).
- `AUTHORITY` defaults to `DOCUMENTED` (kept distinct from `TESTED-*` until probes confirm the documented behaviour).
- COM-bound `VLAX-*` and all `VLA-*` are tagged `(COM VLAX)` or `(COM VLA)` and require `WINDOWS`.
- `VLAX-CURVE-*` are language-level helpers, no platform constraint.
- `VLR-*` are not Windows-bound (reactors run in-process on every supported platform).
- `DOS_*` require `WINDOWS`.
- `ACET-*` / `ACET::*` are tagged `EXPRESS-TOOLS`.
- Entity, table, dictionary, layer-state and related entries are tagged `OBJECTDBX`.
- `GR*` and screen-state functions are tagged `GRAPHICS`.
- DCL functions are tagged `DCL`.
- Interactive prompts are tagged `USER-INPUT`.
- `ARX*` entries are tagged `ARX`. `BRX` classification is reserved for entries surfaced by BricsCAD-specific probes.

The inventory is expected to be refined as probes accumulate. Authorities should rise from `DOCUMENTED` to `TESTED-AUTOCAD`, `TESTED-BRICSCAD`, and ultimately `TESTED-BOTH`. Profile demotions from `STRICT` to vendor-specific occur only when concrete divergence evidence appears.

5 Harness Architecture

The harness is written in pure AutoLISP and lives under `autolisp-test/harness/`:

`rt.lsp` registration core. `deftest`, `deftest-error`, `deftest-skip`, `deftest-versioned` macros, the `*tests*` registry, and the `do-tests`, `do-test`, `do-area`, `do-tag`, `do-profile` runners.

`profiles.lsp` profile and tag definitions, `test-applicable-p`, verdict aggregation.

`platform-detect.lsp` detection of platform and runtime extensions, returning an implementation descriptor (`(impl version platform runtimes profile-target)`).

`report.lsp` canonical s-expression report and human-readable text recap.

`expectations/<impl>/<version>/<host>.lsp` expected-failure overlays, merged at run-time.

`run.lsp` top-level entry point that any implementation can `(load "...")`.

There is no Common Lisp dependency. `clautolisp` is driven by the same `run.lsp` file as AutoCAD and BricsCAD; the only difference is the integration boilerplate around it (the `clautolisp` side adds an ASDF wrapper for SBCL/CCL CI, but the harness itself is unchanged).

6 Test Model

Each `autolisp-test` entry records:

- stable test name,
- language area,
- operator/function under test,
- profile (`STRICT`, `AUTOCAD`, or `BRICSCAD`),
- platform tags,
- runtime tags,
- authority level,

- source evidence reference,
- expected result or expected-error class,
- version qualifiers when behaviour differs across releases.

Expected errors are described in AutoLISP-visible terms (`*error*` invocations, `vl-catch-all-*` catchable structures, exit codes), not in any underlying Common Lisp condition class.

When AutoCAD and BricsCAD diverge, the test is **split** into twin entries:

- one carrying `:profile autocad` with the AutoCAD-attested result;
- one carrying `:profile bricscad` with the BricsCAD-attested result;
- a narrowed `:profile strict` entry, when there is a still-valid common subset.

This keeps every divergence visible by name in the verdict matrix.

7 Report Model

Each run produces a self-contained report under:

- `autolisp-test/results/<impl>/<version>/<host>/<timestamp>/`

Each report contains:

- the implementation descriptor (impl, version, platform, detected runtimes),
- the inventory snapshot version used,
- the verdict matrix, including `CONFORMS`, `DEVIATES (n failing)`, `NOT-APPLICABLE` per subset,
- the per-test result list with status `PASS`, `FAIL`, `XFAIL`, `XPASS`, `SKIP`, `UNIMPLEMENTED`,
- a coverage summary by language area.

The report is produced as a canonical s-expression for diffing, and as a human-readable text recap.

A companion tool (`tools/diff-reports.lisp`) aligns multiple reports and surfaces every divergence with the offending test name and concerned profile/tag combination.

8 First-Version Scope

The first version of `autolisp-test` should be deliberately narrow and defensible.

It starts with:

1. Special forms already implemented in `clautolisp`: `if`, `cond`, `and`, `or`, `setq`, `progn`, `quote`, `while`, `repeat`, `foreach`, `lambda`, `function`, `defun`, `defun-q`.
2. Core list, equality, numeric, string, symbol-handling, and file-I/O functions present in the local specification.
3. AutoLISP-specific behaviour: `defun-q-list-ref`, `defun-q-list-set`, `vl-catch-all-*`, `*error*`, `boundp`, `vl-bb-*`, `vl-doc-*`.
4. Reader-syntax entries.

It defers:

- DCL,
- ActiveX and COM semantics,
- VLR reactor dispatch,
- command-loop integration,
- ARX/BRX runtime extensions,
- Express Tools,
- DOSLib.

Those areas land later, after the harness, the strict-profile core, and the first vendor-divergent twin tests are stable.

9 Phase Outline

The phase letters match the actionable items in `autolisp-test/PLAN.md`:

- **A — Harness:** `rt.lsp`, `profiles.lsp`, `platform-detect.lsp`, `report.lsp`, `run.lsp`, expectations overlay loader.

- **B — Inventory and classification:** completed for the bulk; refinement passes for tag accuracy and authority promotion.
- **C — First corpus, profile STRICT:** special forms, lists, numeric, strings, symbols/errors, file I/O, reader.
- **D — Vendor-divergent twin tests:** printer framing, read/atoi/atof lex models, `or=/=and` rules, command-loop, error mode.
- **E — Tagged coverage:** COM, VLA, VLAX (excl. VLAX-CURVE), VLAX-CURVE, VLR, ARX, BRX, OBJECTDBX, DCL, GRAPHICS, USER-INPUT, EXPRESS-TOOLS, DOSLIB.
- **F — Reporting and CI:** canonical report, diff tool, per-implementation archive, regression rule, `unimplemented` classification.

10 Open Questions

The following points still need explicit project decisions:

1. Should `autolisp-test` also offer an external-driver mode that captures AutoCAD or BricsCAD console output and converts it to a canonical report, or is loading `run.lsp` inside the editor the only supported route?
2. Final shape of vendor-version qualifiers: per-test `:version` metadata, or profile sub-versions like `autocad-2024`, `autocad-2026`, `bricscad-v25`, `bricscad-v26`?
3. Granularity of `BRX` as a single tag, versus splitting per BRX subsystem when probes arrive.
4. Policy for marking tests whose authority cannot rise above `DOCUMENTED` for lack of accessible probe targets (legacy AutoCAD versions, niche host profiles).

11 Standing Recommendation

Begin with:

- the harness shape,

- the inventory classification (already extracted),
- a small corpus for the most central special forms,

and treat every new test as a specification exercise rather than a transliteration of behaviour copied from any other test corpus.